

Estudo Comparativo de Implementações do Quicksort

Pedro Henrique Rodrigues da Silva, Diogo Meireles Ribeiro, Pablo Marcelino Las Cazas

¹Pontifícia Universidade Católica de Minas Gerais

Resumo. *Este trabalho apresenta um estudo comparativo de três implementações do algoritmo Quicksort: a versão recursiva clássica, a versão híbrida com ordenação por inserção para subvetores com menos de M elementos e a versão híbrida com escolha do pivô pela mediana de três. O limiar M foi determinado empiricamente como 8, a partir de vetores aleatórios de 1000 elementos. As versões foram executadas sobre quatro massas de teste (aleatória, ordenada, ordenada inversamente e com muitos elementos repetidos), com múltiplas repetições, contando comparações e trocas e medindo o tempo com o relógio da máquina. Os resultados confirmaram o pior caso quadrático das versões com pivô no último elemento em vetores ordenados e evidenciaram a fragilidade do particionamento de Lomuto diante de chaves duplicadas. A mediana de três mostrou-se a versão mais robusta, mantendo custo $\Theta(n \log n)$ nas massas aleatória, ordenada e inversa, embora seu pior caso teórico continue sendo $\Theta(n^2)$.*

1. Introdução

A ordenação é uma das operações fundamentais da computação, presente em praticamente todos os sistemas que manipulam dados. Entre os algoritmos de ordenação por comparação, o Quicksort se destaca pelo bom desempenho médio, com complexidade $\Theta(n \log n)$, e pelo uso amplo na prática [Ziviani 2018, Sedgwick and Wayne 2011]. Entretanto, o algoritmo apresenta um pior caso quadrático, que depende diretamente da escolha do pivô e da natureza dos dados de entrada [Cormen et al. 2012].

O objetivo deste trabalho é comparar, por meio de experimentos, três versões do Quicksort: a versão recursiva clássica, uma versão híbrida que interrompe a recursão para subvetores com menos de M elementos e os ordena com o algoritmo de ordenação por inserção, e uma versão híbrida com escolha do pivô pela mediana de três elementos. Para cada versão, são avaliados o tempo de execução, o número de comparações e o número de trocas, sobre quatro massas de teste e com múltiplas repetições. Um experimento específico força explicitamente o pior caso do algoritmo, com vetores já ordenados e pivô no último elemento.

O restante deste relatório está organizado da seguinte forma: a Seção 2 apresenta o referencial teórico, com a análise de complexidade dos algoritmos; a Seção 3 descreve a metodologia; a Seção 4 apresenta os resultados e a análise dos testes; a Seção 5 traz a conclusão; e, ao final, são apresentadas as referências bibliográficas.

2. Referencial teórico

2.1. Ordenação por inserção

O algoritmo de ordenação por inserção constrói a sequência ordenada incrementalmente: para cada elemento, as chaves maiores são deslocadas uma posição à direita até que a posição correta seja encontrada [Cormen et al. 2012]. Sua complexidade é $\Theta(n^2)$ no pior

caso e $\Theta(n)$ no melhor caso, quando o vetor já está ordenado. Por esse motivo, o algoritmo é eficiente para vetores pequenos ou quase ordenados, propriedade explorada pelo Quicksort híbrido.

2.2. Quicksort

O Quicksort, proposto por Hoare em 1962 [Hoare 1962], é um algoritmo de ordenação por divisão e conquista: um pivô é escolhido, o vetor é particionado em duas partes, uma com chaves menores ou iguais ao pivô e outra com chaves maiores, e o processo se repete recursivamente sobre cada parte [Ziviani 2018]. No caso médio, a complexidade é $\Theta(n \log n)$ e, na prática, o algoritmo costuma superar os demais métodos de ordenação por comparação [Sedgewick and Wayne 2011]. O particionamento adotado neste trabalho é o de Lomuto, com o pivô no último elemento do intervalo.

2.3. Pior caso do Quicksort

O pior caso do Quicksort ocorre quando as partições ficam completamente desbalanceadas, o que acontece, por exemplo, quando o pivô é sempre o maior ou o menor elemento do intervalo, como em um vetor já ordenado com o pivô no último elemento. Nesse caso, uma das partições fica vazia e o custo total é $\Theta(n^2)$, com exatamente $n(n-1)/2$ comparações [Cormen et al. 2012].

2.4. Estratégias de melhoria

Duas melhorias clássicas foram combinadas neste trabalho. A primeira interrompe a recursão para subvetores com menos de M elementos e os ordena com o algoritmo de inserção, reduzindo o custo das chamadas recursivas [Sedgewick and Wayne 2011]. A segunda escolhe o pivô como a mediana entre o primeiro, o central e o último elemento do intervalo, o que reduz a probabilidade de partições desbalanceadas [Cormen et al. 2012]. Em versões modernas, como o Pattern-defeating Quicksort, essas e outras técnicas são combinadas para lidar com entradas adversas, incluindo vetores com muitas chaves repetidas [Peters 2021]. Outras linhas de pesquisa recentes investigam o uso de dois pivôs [Wild and Nebel 2012] e a redução de falhas de predição de desvio no particionamento [Edelkamp and Weiß 2019].

2.5. Análise de complexidade

Ordenação por inserção. No melhor caso (vetor ordenado) são feitas $n-1$ comparações, ou seja, $\Theta(n)$. No pior caso (vetor inverso), o i -ésimo elemento é comparado com todos os anteriores, totalizando $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} = \Theta(n^2)$. No caso médio, cada elemento é deslocado, em média, por metade dos anteriores, o que dá $\approx n^2/4 = \Theta(n^2)$ [Knuth 1998, Cormen et al. 2012]. O algoritmo é estável, *in-place* e tem custo pequeno por iteração, por isso é vantajoso para subvetores pequenos.

Quicksort. Com particionamento de Lomuto, particionar k elementos custa exatamente $k-1$ comparações. Seu custo é descrito pela recorrência $T(n) = T(q) + T(n-q-1) + \Theta(n)$.

- *Pior caso* ($q = 0$ ou $q = n-1$): $T(n) = T(n-1) + \Theta(n) \Rightarrow \sum_{k=1}^n (k-1) = \frac{n(n-1)}{2} = \Theta(n^2)$.
- *Melhor caso* ($q \approx n/2$): $T(n) = 2T(n/2) + \Theta(n)$. Pelo Teorema Mestre, $a = 2$, $b = 2$, $n^{\log_b a} = n$ e $f(n) = \Theta(n)$ (caso 2), logo $T(n) = \Theta(n \log n)$.

- *Caso médio* (pivô uniformemente distribuído): $C(n) = (n - 1) + \frac{2}{n} \sum_{k=0}^{n-1} C(k)$, cuja solução é $C(n) = 2(n + 1)H_n - 4n \approx 2n \ln n \approx 1,386 n \log_2 n$ [Sedgewick 1978, Knuth 1998].

O espaço adicional é a pilha de recursão: $\Theta(\log n)$ no melhor caso e $\Theta(n)$ no pior.

Mediana de três. Escolher o pivô como a mediana de três elementos reduz o número esperado de comparações para $\approx \frac{12}{7} n \ln n \approx 1,188 n \log_2 n$, cerca de 14% a menos [Sedgewick 1978]. A técnica **não** elimina o pior caso $\Theta(n^2)$: existem entradas adversárias que fazem o pivô ser sempre o segundo menor elemento [Musser 1997, McIlroy 1999].

Chaves repetidas. Com Lomuto e comparação “ $\leq$ ”, um subvetor de m chaves iguais custa $\Theta(m^2)$. Com k valores distintos, cada um com multiplicidade n/k , o custo fica $\approx k \cdot \frac{(n/k)^2}{2} = \frac{n^2}{2k}$. Esse problema motivou o particionamento em três vias [Bentley and McIlroy 1993].

3. Metodologia

3.1. Ambiente de execução

Os experimentos foram executados em um computador com sistema operacional Windows 11 (build 26200), utilizando Python 3.11.9. O tempo de execução foi medido com a função `time.perf_counter`, que utiliza o relógio de maior precisão disponível na máquina, conforme exigido no enunciado. O limite de recursão do interpretador foi elevado para 20000, pois o Quicksort clássico atinge profundidade de recursão proporcional ao tamanho do vetor em seu pior caso.

3.2. Convenções de contagem

Uma comparação foi contabilizada a cada comparação entre dois elementos. Uma troca foi contabilizada a cada troca de posição entre dois elementos; na ordenação por inserção, cada deslocamento de um elemento para a direita foi contabilizado como uma troca, e a escrita da chave em sua posição final não foi contabilizada. Na escolha do pivô pela mediana de três, as três comparações e as eventuais trocas realizadas na escolha do pivô foram contabilizadas, assim como a movimentação do pivô para o último elemento antes da partição. Ressalta-se que autotrocas (quando um elemento é trocado por si mesmo) não foram contabilizadas, o que explica a ausência de trocas do Quicksort recursivo e do híbrido na massa ordenada.

3.3. Massas de teste

Quatro massas de teste foram geradas com semente fixa, garantindo que todas as versões e todas as repetições utilizassem exatamente os mesmos dados:

- **aleatória:** valores sorteados uniformemente entre 0 e 9999999;
- **ordenada:** valores em ordem crescente;
- **inversa:** valores em ordem decrescente;
- **com muitos repetidos:** valores sorteados entre 0 e 99, com alta frequência de repetições.

Para as massas ordenada e inversa, que induzem o pior caso das versões com pivô no último elemento, foram utilizados tamanhos menores (1000, 2000 e 4000 elementos), tanto pelo custo quadrático quanto pela profundidade de recursão proporcional a n imposta pelo interpretador Python. As massas aleatória e de repetidos utilizaram 1000, 10000 e 50000 elementos.

3.4. Determinação empírica do parâmetro M

O valor de M foi determinado empiricamente executando o Quicksort híbrido sobre vetores aleatórios de 1000 elementos, tamanho sugerido no enunciado, com 100 repetições para cada candidato de M no conjunto $\{5, 8, 10, 12, 15, 20, 25, 30, 40, 50\}$. A escolha foi confirmada com vetores de 10000 elementos e 30 repetições. O valor adotado foi $M = 8$.

3.5. Execução dos experimentos

Cada versão foi executada múltiplas vezes sobre cópias idênticas dos dados: 5 repetições para as massas aleatória e de repetidos, 3 para as massas ordenada e inversa e 2 para o experimento do pior caso, que utilizou vetores ordenados de 1000 a 8000 elementos. Para cada repetição foram registrados o tempo, o número de comparações e o número de trocas, e os resultados são apresentados como médias. As tabelas completas de resultados foram gravadas em arquivos CSV na pasta `resultados/tabelas`, e as figuras, na pasta `resultados/figuras`, ambas geradas pelo caderno de análises do projeto (`notebooks/analise_quicksort.ipynb`).

4. Resultados e análise

4.1. Determinação empírica do parâmetro M

A Tabela 1 apresenta os resultados da determinação de M com vetores de 1000 elementos e 100 repetições. O menor tempo médio foi observado para $M = 8$, com aproximadamente 5,85 ms. Entre $M = 5$ e $M = 12$, porém, os tempos ficam entre 5,8 e 7,1 ms, diferenças comparáveis à variabilidade das medições. A partir de $M = 15$, o tempo cresce de forma consistente, como mostra a Figura 1. O número de comparações cresce de forma monotônica com M , de 11.974 comparações para $M = 5$ até 16.481 para $M = 50$, confirmando que a escolha de M envolve um equilíbrio entre o custo das chamadas recursivas e o custo da ordenação por inserção.

Tabela 1. Determinação de M com vetores aleatórios de 1000 elementos e 100 repetições.

M	Tempo médio (ms)	Comparações médias	Trocas médias
5	6.0701	11 974	5795
8	5.8473	12 088	6045
10	7.0567	12 136	6189
12	6.9552	12 256	6431
15	8.6985	12 460	6759
20	8.4578	12 771	7283
25	12.0830	13 312	7967
30	12.5365	14 008	8871
40	12.7601	15 217	10 295
50	15.2614	16 481	11 727

Na confirmação com vetores de 10000 elementos e 30 repetições, apresentada na Tabela 2, o mínimo observado foi $M = 12$. Os tempos de $M = 5$ e $M = 8$ (271,9

e 301,2 ms) destoam dos demais, que ficam entre 77 e 132 ms, embora os números de comparações sejam praticamente iguais (146.769 para $M = 8$ e 148.733 para $M = 12$). Como as comparações não justificam uma diferença dessa ordem, ela indica interferência na medição de tempo, e não um efeito de M . O valor adotado nos demais experimentos foi $M = 8$, coerente com a faixa tipicamente recomendada na literatura, que situa o limiar entre 5 e 20 elementos.

Tabela 2. Confirmação da determinação de M com vetores aleatórios de 10000 elementos e 30 repetições.

M	Tempo médio (ms)	Comparações médias	Trocas médias
5	271.8502	146 151	71 588
8	301.2391	146 769	74 046
10	115.3996	147 456	75 857
12	77.5156	148 733	78 348
15	103.6329	150 515	81 479
20	93.4720	154 651	87 589
25	109.6977	159 841	94 451
30	123.8841	164 135	100 162
40	125.5155	177 405	116 104
50	131.1272	190 107	130 925

A Figura 1 ilustra o comportamento do tempo médio em função de M para os vetores de 1000 elementos.

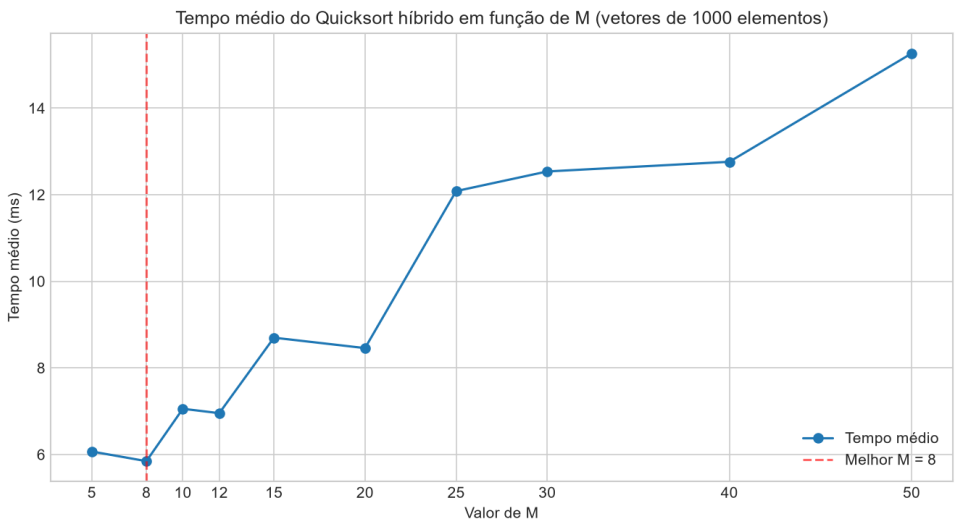


Figura 1. Tempo médio do Quicksort híbrido em função de M , com vetores de 1000 elementos.

4.2. Massa aleatória

A Tabela 3 e a Figura 2 apresentam os resultados da massa aleatória. As três versões apresentam comportamento compatível com a complexidade $\Theta(n \log n)$. Para 50000 elementos, os tempos médios foram 0,4959 s (recursivo), 0,4742 s (híbrido) e 0,4448 s (mediana

de três), com 927.813, 931.395 e 832.234 comparações, respectivamente. A mediana de três realizou cerca de 10% menos comparações, porque o pivô escolhido tende a dividir o vetor de forma mais equilibrada, e essa redução compensa as três comparações adicionais realizadas na escolha do pivô. Para $n = 10000$, a mediana de três mostrou-se mais lenta (0,1060 s contra 0,0629 s do recursivo), evidenciando a alta variabilidade das medições nesta escala, embora para $n = 50000$ ela tenha sido a mais rápida. Além disso, para 50.000 elementos, as 927.813 comparações observadas no recursivo ficam a apenas -1,3% da previsão teórica matemática (939.723), confirmando o modelo.

Tabela 3. Massa aleatória, com 5 repetições.

Tamanho	Versão	Tempo médio (s)	Comparações médias	Trocas médias
1000	Recursivo	0.0105	11 972	5715
1000	Híbrido ($M = 8$)	0.0063	12 088	6045
1000	Mediana de três ($M = 8$)	0.0046	10 320	5263
10 000	Recursivo	0.0629	154 959	70 735
10 000	Híbrido ($M = 8$)	0.0751	155 673	73 897
10 000	Mediana de três ($M = 8$)	0.1060	137 460	67 641
50 000	Recursivo	0.4959	927 813	456 100
50 000	Híbrido ($M = 8$)	0.4742	931 395	472 384
50 000	Mediana de três ($M = 8$)	0.4448	832 234	408 948

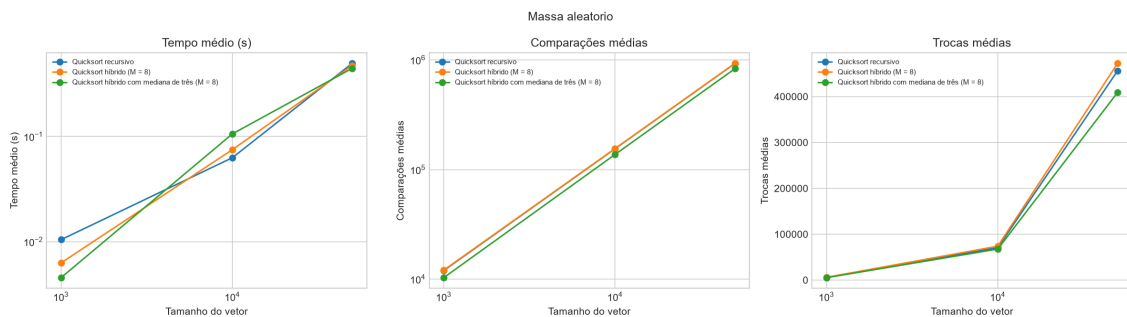


Figura 2. Resultados da massa aleatória: tempo médio, comparações médias e trocas médias.

4.3. Massa ordenada

A Tabela 4 e a Figura 3 apresentam os resultados da massa ordenada. O Quicksort recursivo realizou exatamente $n(n-1)/2$ comparações, com 499.500 comparações para 1000 elementos, 1.999.000 para 2000 e 7.998.000 para 4000, e o híbrido realizou 15 comparações a menos em cada tamanho, confirmando o comportamento quadrático previsto pela teoria. O número de trocas foi zero, pois o pivô, sendo sempre o maior elemento, já se encontra em sua posição final. Apesar das comparações exatas, as razões de tempo variaram entre $3,5\times$ e $6,6\times$ no trecho de 2000 para 4000, refletindo a instabilidade típica das medições. Além disso, a massa ordenada foi mais lenta que a inversa, provavelmente pelo custo das chamadas à função de troca nas autotrocas, que são executadas mesmo sem serem contabilizadas. No vetor ordenado, todo elemento de cada partição é menor

ou igual ao pivô e gera uma chamada, cerca de $n^2/2$ no total (500.499 para 1000 elementos), enquanto no vetor inverso isso ocorre em apenas metade das partições, cerca de $n^2/4$ chamadas (250.499 para 1000 elementos).

A mediana de três manteve o comportamento $\Theta(n \log n)$: para 4000 elementos, o tempo médio foi 0,0228 s contra 10,8501 s do recursivo, cerca de 476 vezes mais rápido, com 39.497 comparações contra 7.998.000. A escolha do pivô pela mediana elimina as partições desbalanceadas, pois o pivô passa a ser o elemento central do intervalo.

Tabela 4. Massa ordenada, com 3 repetições.

Tamanho	Versão	Tempo médio (s)	Comparações médias	Trocas médias
1000	Recursivo	0.2989	499 500	0
1000	Híbrido ($M = 8$)	0.2845	499 485	0
1000	Mediana de três ($M = 8$)	0.0019	7879	254
2000	Recursivo	1.6344	1 999 000	0
2000	Híbrido ($M = 8$)	2.4903	1 998 985	0
2000	Mediana de três ($M = 8$)	0.0105	17 752	510
4000	Recursivo	10.8501	7 998 000	0
4000	Híbrido ($M = 8$)	8.6474	7 997 985	0
4000	Mediana de três ($M = 8$)	0.0228	39 497	1022

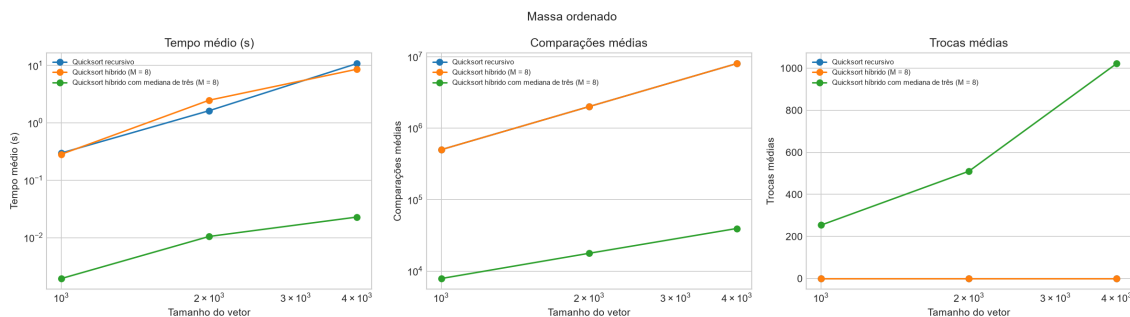


Figura 3. Resultados da massa ordenada: tempo médio, comparações médias e trocas médias.

4.4. Massa ordenada inversamente

A Tabela 5 e a Figura 4 apresentam os resultados da massa inversa. O comportamento é análogo ao da massa ordenada: o Quicksort recursivo realizou exatamente $n(n-1)/2$ comparações e o híbrido, 5 a menos, com custo quadrático, enquanto a mediana de três manteve o custo $\Theta(n \log n)$. A diferença em relação à massa ordenada está no número de trocas. Na massa inversa, o pivô alterna entre o menor e o maior elemento do intervalo, gerando uma única troca efetiva a cada par de partições, o que totaliza $\lfloor n/2 \rfloor$ trocas no Quicksort recursivo (500, 1000 e 2000 trocas para os três tamanhos). Na mediana de três, o número de trocas é maior, pois inclui as trocas oriundas de partições mais equilibradas (onde mais elementos de fato mudam de lado), e não apenas da escolha do pivô, ainda que com muito menos comparações.

Tabela 5. Massa ordenada inversamente, com 3 repetições.

Tamanho	Versão	Tempo médio (s)	Comparações médias	Trocas médias
1000	Recursivo	0.1400	499 500	500
1000	Híbrido ($M = 8$)	0.1787	499 495	512
1000	Mediana de três ($M = 8$)	0.0085	15 373	6792
2000	Recursivo	0.5882	1 999 000	1000
2000	Híbrido ($M = 8$)	0.7281	1 998 995	1012
2000	Mediana de três ($M = 8$)	0.0132	35 205	15 324
4000	Recursivo	2.7175	7 998 000	2000
4000	Híbrido ($M = 8$)	4.2117	7 997 995	2012
4000	Mediana de três ($M = 8$)	0.0369	79 969	34 426

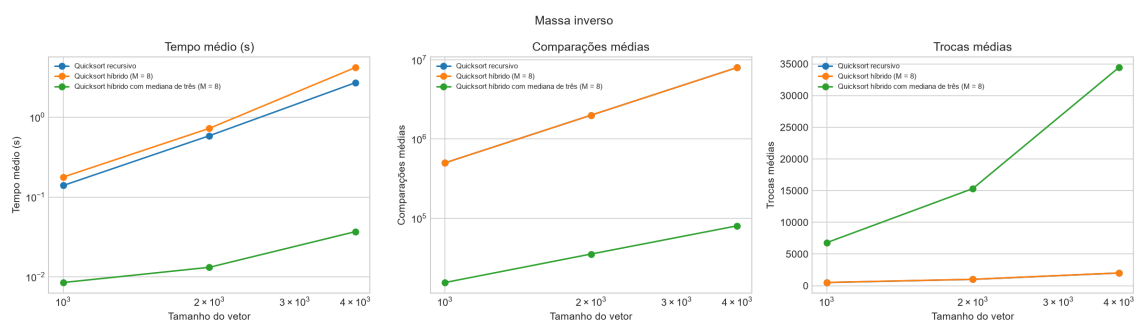


Figura 4. Resultados da massa ordenada inversamente: tempo médio, comparações médias e trocas médias.

4.5. Massa com muitos elementos repetidos

A Tabela 6 e a Figura 5 apresentam os resultados da massa com muitos repetidos, que revelou o ponto mais frágil das versões com particionamento de Lomuto diante de chaves duplicadas. Com chaves sorteadas entre 0 e 99, a ordenação de 50000 elementos exigiu cerca de 12.929.058 comparações e 8,9913 s no Quicksort recursivo, contra cerca de 927.813 comparações e 0,4959 s na massa aleatória. O particionamento de Lomuto com comparação menor ou igual move todas as chaves iguais ao pivô para um único lado da partição; quando um subvetor fica formado por chaves iguais, ele é reduzido de um elemento por partição, comportamento análogo ao pior caso para cada grupo de chaves repetidas. A mediana de três não elimina esse efeito e ainda acrescenta o custo da escolha do pivô. Esse resultado reforça a recomendação de esquemas de particionamento que distribuem as chaves iguais entre os dois lados, como a variante de Hoare [Hoare 1962]. O custo observado de 12,9 milhões de comparações ($n = 50000, k = 100$) está bastante alinhado à previsão teórica de $\approx n^2/(2k)$, que resulta em 12,5 milhões (erro de +3,4%).

Tabela 6. Massa com muitos elementos repetidos, com 5 repetições.

Tamanho	Versão	Tempo médio (s)	Comparações médias	Trocas médias
1000	Recurso	0.0057	12 474	3795
1000	Híbrido ($M = 8$)	0.0066	11 177	3795
1000	Mediana de três ($M = 8$)	0.0048	11 544	4481
10 000	Recurso	0.3541	584 998	48 401
10 000	Híbrido ($M = 8$)	0.3379	583 498	48 401
10 000	Mediana de três ($M = 8$)	0.3768	598 536	49 858
50 000	Recurso	8.9913	12 929 058	146 354
50 000	Híbrido ($M = 8$)	8.6631	12 927 558	146 354
50 000	Mediana de três ($M = 8$)	9.1529	13 042 342	235 978

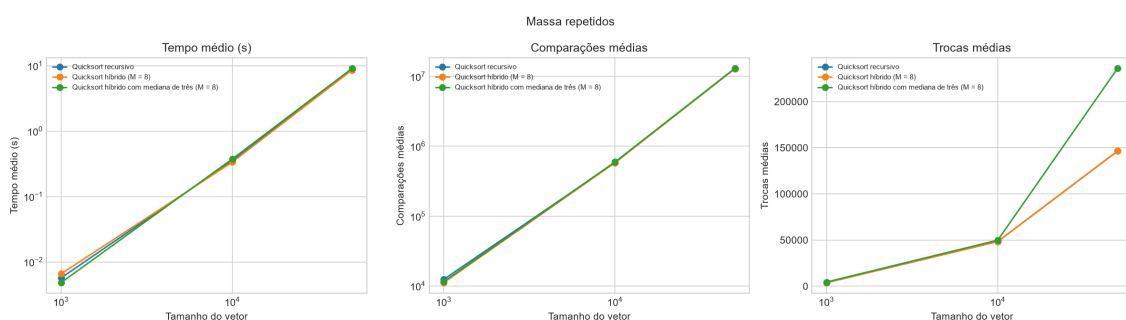


Figura 5. Resultados da massa com muitos elementos repetidos: tempo médio, comparações médias e trocas médias.

4.6. Pior caso forçado

A Tabela 7 apresenta os resultados do experimento que força explicitamente o pior caso, com vetores já ordenados de 1000 a 8000 elementos e 2 repetições. O Quicksort recursivo apresentou razão de comparações de exatamente 4,00 entre tamanhos consecutivos, o que caracteriza o custo quadrático, e as razões de tempo ficaram próximas de 4, com a variabilidade esperada de medição. A mediana de três apresentou razões de comparações próximas de 2,25 e razões de tempo entre 2,1 e 3,0, compatíveis com $\Theta(n \log n)$, e foi cerca de 622 vezes mais rápida que o recursivo no vetor de 8000 elementos, com 0,0344 s contra 21,3904 s. Os tamanhos desse experimento foram limitados a 8000 elementos pelo custo quadrático e pela profundidade de recursão, conforme justificado na metodologia.

Tabela 7. Pior caso forçado: vetor já ordenado com pivô no último elemento, 2 repetições.

Tamanho	Versão	Tempo médio (s)	Comparações médias	Trocas médias
1000	Recursivo	0.3815	499 500	0
1000	Híbrido ($M = 8$)	0.3029	499 485	0
1000	Mediana de três ($M = 8$)	0.0020	7879	254
2000	Recursivo	1.1340	1 999 000	0
2000	Híbrido ($M = 8$)	1.2002	1 998 985	0
2000	Mediana de três ($M = 8$)	0.0059	17 752	510
4000	Recursivo	4.8139	7 998 000	0
4000	Híbrido ($M = 8$)	6.5423	7 997 985	0
4000	Mediana de três ($M = 8$)	0.0163	39 497	1022
8000	Recursivo	21.3904	31 996 000	0
8000	Híbrido ($M = 8$)	26.1522	31 995 985	0
8000	Mediana de três ($M = 8$)	0.0344	86 986	2046

As Tabelas 8 e 9 apresentam as razões entre os resultados de tamanhos consecutivos, ao dobrar o tamanho do vetor. As razões próximas de 4 para o tempo e exatamente 4 para as comparações confirmam o crescimento quadrático do Quicksort recursivo e do híbrido, enquanto as razões de comparações da mediana de três, entre 2,20 e 2,25, coincidem com o valor esperado para o crescimento $\Theta(n \log n)$ ao dobrar o tamanho, $2 \log(2n) / \log n \approx 2,20$ para $n = 1000$. A Figura 6 ilustra o comportamento das três versões nesse experimento.

Tabela 8. Razão entre os tempos médios ao dobrar o tamanho do vetor.

Versão	Faixa de tamanhos	Razão
Recursivo	1000 para 2000	2.97
Recursivo	2000 para 4000	4.25
Recursivo	4000 para 8000	4.44
Híbrido ($M = 8$)	1000 para 2000	3.96
Híbrido ($M = 8$)	2000 para 4000	5.45
Híbrido ($M = 8$)	4000 para 8000	4.00
Mediana de três ($M = 8$)	1000 para 2000	2.97
Mediana de três ($M = 8$)	2000 para 4000	2.78
Mediana de três ($M = 8$)	4000 para 8000	2.11

Tabela 9. Razão entre as comparações médias ao dobrar o tamanho do vetor.

Versão	Faixa de tamanhos	Razão
Recursivo	1000 para 2000	4.00
Recursivo	2000 para 4000	4.00
Recursivo	4000 para 8000	4.00
Híbrido ($M = 8$)	1000 para 2000	4.00
Híbrido ($M = 8$)	2000 para 4000	4.00
Híbrido ($M = 8$)	4000 para 8000	4.00
Mediana de três ($M = 8$)	1000 para 2000	2.25
Mediana de três ($M = 8$)	2000 para 4000	2.22
Mediana de três ($M = 8$)	4000 para 8000	2.20

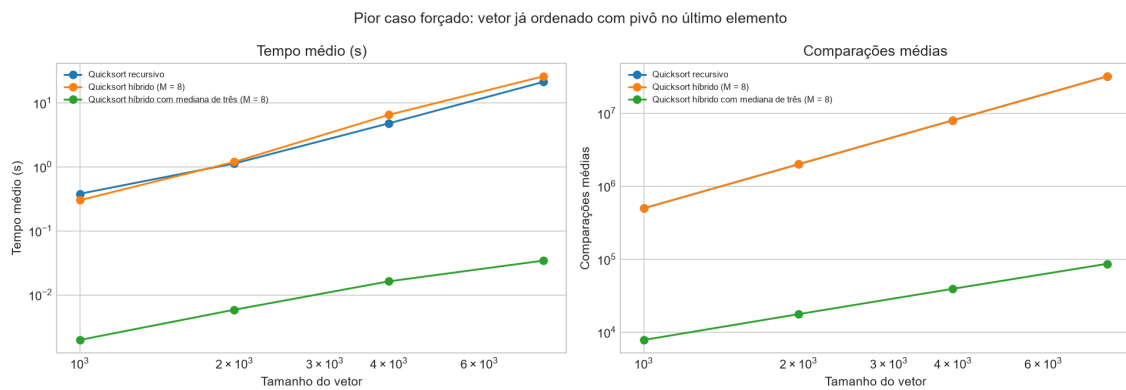


Figura 6. Pior caso forçado: tempo médio e comparações médias em vetores já ordenados.

5. Conclusão

Este trabalho implementou e comparou três versões do Quicksort, determinando empiricamente o limiar $M = 8$ e avaliando as versões sobre quatro massas de teste, com múltiplas repetições e contagem de comparações e trocas. Os experimentos confirmaram que a escolha do pivô é o fator mais decisivo para a robustez do Quicksort: a mediana de três eliminou, ao custo de um pequeno overhead, o pior caso para entradas ordenadas e inversas, embora seu pior caso teórico continue quadrático, enquanto a interrupção com ordenação por inserção, embora benéfica em termos de constantes, não altera a complexidade do algoritmo. Na massa ordenada, o híbrido poupou apenas 15 comparações em relação ao recursivo, exatamente o custo evitado pela ordenação por inserção nos subvetores terminais já ordenados.

A massa com muitos repetidos evidenciou a necessidade de um particionamento que distribua as chaves iguais entre os dois lados, tema de variantes modernas do algoritmo. Como limitações do estudo, destaca-se a variabilidade das medições de tempo para vetores pequenos e a restrição dos tamanhos das massas ordenada e inversa, imposta pelo custo quadrático e pela profundidade de recursão do Python. Os tempos absolutos não devem ser comparados com outras linguagens, pois Python é uma linguagem interpre-

tada; as conclusões se baseiam nas tendências relativas entre as versões e na evolução dos tempos com o tamanho dos vetores.

Referências

- Bentley, J. L. and McIlroy, M. D. (1993). Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2012). *Algoritmos: Teoria e Prática*. Elsevier, Rio de Janeiro, 3 edition.
- Edelkamp, S. and Weiß, A. (2019). BlockQuicksort: Avoiding branch mispredictions in Quicksort. *ACM Journal of Experimental Algorithmics*, 24:1.4:1–1.4:22.
- Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1):10–15.
- Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2 edition.
- McIlroy, M. D. (1999). A killer adversary for Quicksort. *Software: Practice and Experience*, 29(4):341–344.
- Musser, D. R. (1997). Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993.
- Peters, O. R. L. (2021). Pattern-defeating Quicksort. *arXiv preprint arXiv:2106.05123*. Acesso em: 17 ago. 2026.
- Sedgewick, R. (1978). Implementing Quicksort programs. *Communications of the ACM*, 21(10):847–857.
- Sedgewick, R. and Wayne, K. (2011). *Algorithms*. Addison-Wesley, Boston, 4 edition.
- Wild, S. and Nebel, M. E. (2012). Average case analysis of Java 7’s dual pivot Quicksort. In *Algorithms – ESA 2012*, volume 7501 of *Lecture Notes in Computer Science*, pages 825–836. Springer.
- Ziviani, N. (2018). *Projeto de Algoritmos: com Implementações em Pascal e C*. Cengage Learning, Porto Alegre, 3 edition.